Automated Real-Time Hydroponics Telemetry and Alerting System

Nitish Kumar Sah

123BT0791

1. Project Overview & Objectives

The objective of this research project is to design, develop, and deploy an automated Internet of Things (IoT) monitoring system for a hydroponic environment. Hydroponic agriculture requires precise control over biological and environmental parameters. This project eliminates manual data collection by establishing a continuous, real-time data pipeline from submerged digital sensors to a centralized, cloud-hosted dashboard.

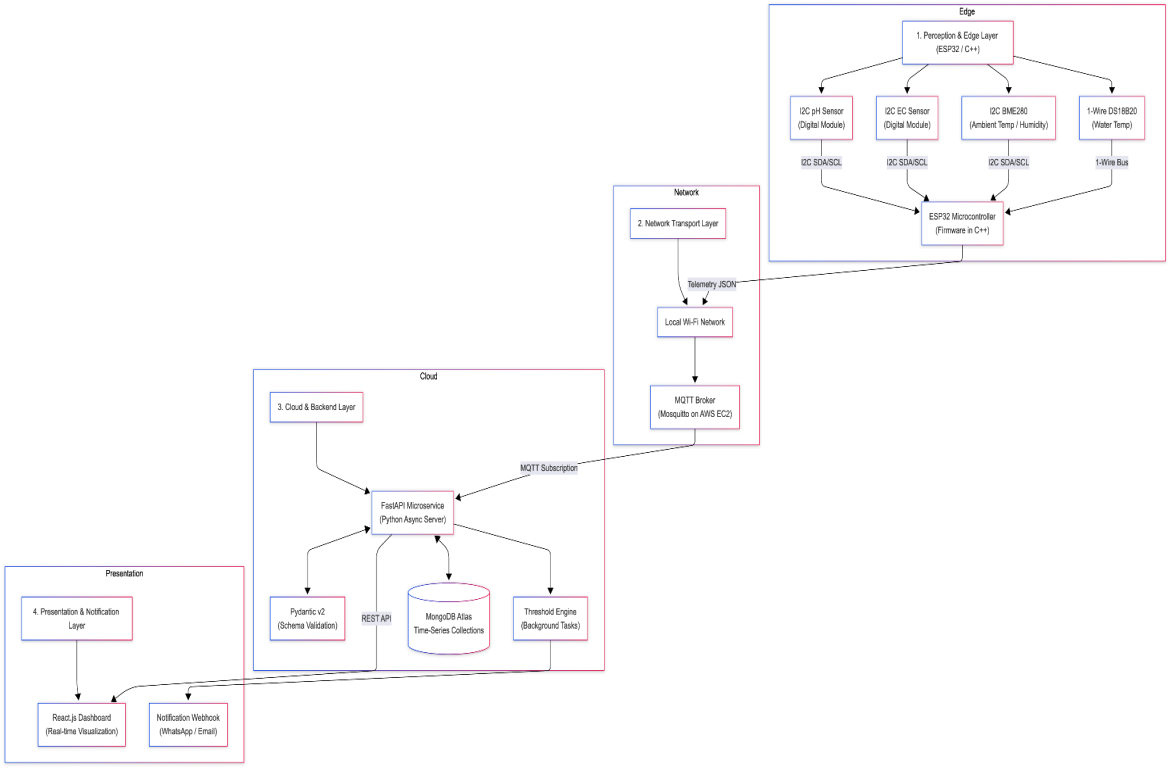
Core Deliverables:

- A hardware edge device using precision ADC conditioning to minimize analog noise interference.
- A scalable, microservice-based backend to ingest, validate, and store high-frequency time-series data.
- A real-time web dashboard for visualization and data analysis.
- An automated alerting engine that triggers immediate notifications when critical biological parameters (e.g., pH, electrical conductivity, temperature) breach predefined safety thresholds.

2. System Architecture & Tech Stack

The system is divided into four distinct layers, utilizing industry-standard communication protocols and software frameworks:

- **Hardware Layer (ESP32 & C++):** An ESP32 microcontroller utilizing the I2C and 1-Wire protocols to aggregate data from local digital sensor modules.
- **Transport Layer (MQTT):** A lightweight Publish/Subscribe messaging protocol (e.g., Eclipse Mosquitto) ensures reliable, low-latency telemetry transmission from.
- **Backend & Database Layer (GoLang & MySQL):** A high-performance RESTful API built with GoLang will consume the MQTT data stream. Data payloads will be securely stored in a MySQL database, optimized for efficient storage, retrieval, and analysis of time-stamped telemetry data.
- **Presentation Layer (React.js):** A dynamic, responsive frontend providing real-time analytical charts and system status overviews.



3. Hardware Requirements

To ensure data accuracy while optimizing for cost and lab availability, this project utilizes analog sensors conditioned via a 16-bit external ADC (ADS1115) with I2C output. This approach effectively mitigates electromagnetic interference (EMI) common in hydroponic setups, serving as a practical alternative to more expensive integrated digital pH/EC modules.

Component	**Protocol / Interface**	**Purpose**
ESP32 Development Board	Wi-Fi / I2C / 1-Wire	Core processing, sensor polling, and network transmission over MQTT.
ADS1115 16-bit ADC Module	I2C	External 4-channel ADC; digitizes analog pH/EC voltages into high-resolution I2C signals for the ESP32.
DFRobot Analog pH Sensor Kit V2	Analog (to ADS1115)	Measures nutrient solution acidity; outputs analog voltage to the external ADC.
DFRobot Analog EC Sensor Kit V2 (K=1.0)	Analog (to ADS1115)	Measures electrical conductivity (nutrient density); outputs analog voltage to the external ADC.
DS18B20 Waterproof Probe	1-Wire	Submerged water temperature monitoring.
BME280 Module	I2C	Ambient room temperature, humidity, and barometric pressure.
Misc. Electronics	N/A	Breadboard, jumper wires, 4.7kΩ resistors, pH/EC calibration buffers.

4. Sprint-Wise Execution Plan

The project will be executed in four distinct, two-week sprints.

Sprint 1: Edge Hardware Integration & Calibration (Weeks 1-4)

- Procure all listed hardware and chemical calibration solutions.
- Calibrate the pH and EC probes to baseline standards.
- Wire the I2C bus (pH, EC, BME280) and 1-Wire bus (DS18B20) to the ESP32.
- Develop C++ firmware to successfully poll digital addresses, read the float values, and structure them into a cohesive JSON payload.

Sprint 2: Network & Message Broker Setup (Weeks 5-8)

- Deploy an MQTT broker to handle incoming telemetry.
- Program the ESP32 to authenticate with the local Wi-Fi and publish the JSON payload to a dedicated MQTT topic at 60-second intervals.
- Establish the MySQL database and configure the required tables, indexes, and schema optimized for rapid insertion and retrieval of time-stamped telemetry data.

Sprint 3: API & Database Engineering (Weeks 9-11)

- Develop the GoLang backend service to subscribe to the MQTT topic.
- Implement data validation (e.g., using Go structs and validation libraries) to ensure incoming payloads are structurally sound before executing database insertion.
- Design and optimize the MySQL database schema for efficient storage and retrieval of time-stamped telemetry data.
- Develop secure REST endpoints (GET requests with pagination) allowing the frontend to retrieve historical data efficiently.

Sprint 4: Frontend Dashboard & Alerting System (Weeks 12-14)

- Develop the React.js web interface, integrating a charting library to visualize sensor data over customizable timeframes.
- Implement the alerting logic within the GoLang backend service. Define biological safety thresholds for all monitored metrics.
- Integrate an external notification service (e.g., SMTP email or Telegram API) to push immediate alerts to the user when thresholds are violated.
- Final system testing, bug fixing, and project documentation.